

API Security Risk Analysis

Target API: JSONPlaceholder (<https://jsonplaceholder.typicode.com>)

Scope: Public / Demo REST API | Method: Read-Only Analysis

Prepared by:	Steven Anthony
Internship Program:	Future Interns – Cyber Security Track
Repository:	FUTURE_CS_03
Report Date:	May 24, 2026
Tools Used:	Postman, Browser DevTools, OWASP API Top 10

Disclaimer: This report is prepared for educational purposes as part of a cyber security internship. All testing was conducted on public demo APIs that are specifically designed for testing and learning. No real systems were accessed, exploited, or harmed in any way.



Table of Contents

1. Introduction & Scope
 2. About JSONPlaceholder API
 3. Methodology
 4. Endpoints Tested
 5. Security Risk Findings
 6. Risk Summary Table
 7. Business Impact Analysis
 8. Remediation Recommendations
 9. OWASP API Top 10 Mapping
 10. Conclusion
 11. References
-

1 Introduction & Scope

This report is submitted as part of Task 3 of the Future Interns Cyber Security Internship. The goal of this task was to perform a read-only API security risk analysis on a public demo API, document the findings in a structured format, and suggest practical remediation steps — similar to how security consultants deliver API audit reports to SaaS clients.

APIs (Application Programming Interfaces) are the backbone of modern software. Most mobile apps, web dashboards, and SaaS products use APIs to communicate between services. When APIs are not secured properly, they can expose sensitive data, allow unauthorized access, or be abused in ways that affect the entire platform.

Scope of this Analysis: This analysis is limited to the JSONPlaceholder public demo API (<https://jsonplaceholder.typicode.com>). Only read-only GET requests were made. No exploitation, brute force, fuzzing, or flooding of any kind was attempted. All observations are based on API responses, header inspection, and documentation review.

What this report covers:

- Endpoint structure and access control assessment
- Authentication and authorization analysis
- Data exposure risks in API responses
- Missing security controls (rate limiting, input validation)
- Business impact of identified risks
- Practical remediation suggestions for each finding

2 About JSONPlaceholder API

JSONPlaceholder is a free, public REST API maintained by Typicode. It is widely used by developers to test frontend applications, practice API calls, and build demo projects. It serves approximately 3 billion requests per month and exposes several resource endpoints covering posts, comments, users, albums, photos, and todos.

API Name	JSONPlaceholder
Base URL	https://jsonplaceholder.typicode.com
Type	Public REST API (Demo / Test)
Maintained by	Typicode (typicode.com)
Authentication Required	No – fully open / unauthenticated
HTTP Methods Supported	GET, POST, PUT, PATCH, DELETE
Resources Available	/posts, /comments, /users, /albums, /photos, /todos
Total Records	~5,900+ across all resources

Note: JSONPlaceholder does not persist any data — all write requests (POST, PUT, DELETE) return fake success responses without modifying any actual data. This makes it safe for security testing practice without any risk of real data impact.

3 Methodology

The analysis followed a structured, consultant-style approach based on the OWASP API Security Top 10 framework. The process involved three main phases: reconnaissance, inspection, and documentation.

Step 1 – Documentation Review

Reviewed the official JSONPlaceholder documentation and resource guide available at jsonplaceholder.typicode.com/guide to understand available endpoints, supported HTTP methods, and the structure of responses.

Step 2 – Endpoint Testing with Postman

Sent GET requests to all major endpoints using Postman. Captured response headers, status codes, response body structure, and response time. Noted which endpoints require authentication and which are publicly accessible without any token or key.

Step 3 – Header & Response Inspection

Inspected HTTP response headers using both Postman and Browser DevTools (F12 > Network tab). Checked for security headers like X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, and Strict-Transport-Security.

Step 4 – Risk Identification

Compared observed API behaviour against the OWASP API Security Top 10 checklist (github.com/OWASP/API-Security) and the API Security Checklist by Shieldfy (github.com/shieldfy/API-Security-Checklist) to identify gaps and risks.

Step 5 – Report Documentation

Classified each finding by severity (High / Medium / Low / Info), described the business impact in simple language, and wrote clear, actionable remediation suggestions.

Ethical Statement: All testing was strictly read-only. No authentication bypass, injection attacks, rate-limit abuse, or exploitation was attempted at any point. This analysis is conducted purely for educational purposes as part of a cyber security internship program.

4 Endpoints Tested

The following endpoints were tested during this analysis using Postman. All requests were standard GET requests with no custom headers, tokens, or authentication.

Endpoint	Method	Status	Auth Required	Notes
/posts	GET	200 OK	No	Returns 100 post objects
/posts/1	GET	200 OK	No	Single post with userId, id, title, body
/comments	GET	200 OK	No	Returns 500 comments with email fields
/users	GET	200 OK	No	Full user PII: name, email, address, phone, company
/users/1	GET	200 OK	No	Individual user with geo coordinates
/albums	GET	200 OK	No	100 album records
/photos	GET	200 OK	No	5000 photo records with thumbnail URLs
/todos	GET	200 OK	No	200 todo items with completion status
/posts/9999	GET	404	No	Non-existent resource – error handling test
/users?id=1	GET	200 OK	No	Query filter – returns filtered user data

Screenshot reference: See /screenshots/ folder in the GitHub repository for Postman captures of each endpoint request and response.

5 Security Risk Findings

The following findings were identified during the analysis. Each finding is described with its observation, the associated risk, potential business impact, and a recommended fix. Findings are numbered and classified by severity.

F-01	No Authentication on Any Endpoint	HIGH
OWASP Reference	API2:2023 – Broken Authentication	
Observation	Every endpoint on the JSONPlaceholder API is fully accessible without any authentication. There are no API keys, bearer tokens, OAuth flows, or session cookies required. Any person or automated script on the internet can access any resource at any time — including endpoints that expose user data.	
Evidence	GET /users → 200 OK (no Authorization header sent)	
Business Impact	In a real API, having no authentication means any attacker — or even a curious user — can read all data in the system without proving who they are. This would be a critical issue for any production SaaS product.	
Remediation	Implement token-based authentication (JWT or OAuth 2.0) for all endpoints. Sensitive endpoints like /users should always require a valid access token. Use an API gateway to enforce authentication across all routes.	

F-02	Excessive Data Exposure – /users Endpoint	HIGH
OWASP Reference	API3:2023 – Broken Object Property Level Authorization	
Observation	The /users endpoint returns a large amount of user information in a single response without any filtering. Each user object contains: full name, username, email address, complete physical address (street, suite, city, zip), phone number, company name, and geolocation coordinates (latitude & longitude). Most of this data is unnecessary for typical API consumers.	
Evidence	GET /users/1 returns: name, username, email, address.street, address.geo.lat, address.geo.lng, phone, website, company.name, company.catchPhrase	
Business Impact	Exposing this level of detail — especially geo coordinates and phone numbers — in a production system would violate GDPR and other data privacy regulations. Attackers could harvest this information for phishing, social engineering, or physical targeting of individuals.	
Remediation	Apply response filtering — return only the fields the client actually needs. Use field-level access control (e.g., only return email for authenticated users, never return geo coordinates via a public endpoint). Follow the principle of data minimization.	

F-03	Email Addresses Exposed in /comments Endpoint	MEDIUM
OWASP Reference	API3:2023 – Broken Object Property Level Authorization	
Observation	The /comments endpoint returns 500 records, each containing a name and email field associated with a commenter. These email addresses are exposed in plain text to any unauthenticated caller. Running a single GET request returns all 500 email values at once.	
Evidence	GET /comments → 500 records, each with email field in plaintext	
Business Impact	If this were a real platform, bulk harvesting of email addresses through the comments API could be used for spam campaigns, credential stuffing attacks, or targeted phishing. Email addresses are considered personal data under most privacy frameworks.	
Remediation	Mask or omit email addresses from public-facing API responses. If email must be returned, restrict this to authenticated and authorized users only. Implement pagination to limit how much data is returned per request.	

F-04	No Rate Limiting Detected	MEDIUM
OWASP Reference	API4:2023 – Unrestricted Resource Consumption	
Observation	During testing, repeated requests were sent to the same endpoints in quick succession (10+ requests in under 5 seconds). No rate limiting was observed — all requests returned 200 OK without any throttling, 429 Too Many Requests response, or CAPTCHA challenge. No X-RateLimit headers were present in responses.	
Evidence	No 429 status, no Retry-After header, no X-RateLimit-Limit or X-RateLimit-Remaining in response headers after repeated requests	
Business Impact	Lack of rate limiting makes the API vulnerable to denial-of-service (DoS) attacks through request flooding, automated scraping of all data, and brute force attacks on any auth endpoints. Even without malicious intent, a poorly written client application could accidentally flood the server.	
Remediation	Implement rate limiting at the API gateway level — for example, 100 requests per minute per IP. Return 429 Too Many Requests with a Retry-After header when limits are exceeded. Consider different limits for authenticated vs anonymous users.	

F-05	Missing Security Headers in HTTP Responses	MEDIUM
OWASP Reference	API7:2023 – Server-Side Request Forgery (SSRF) / Misconfiguration	
Observation	Inspection of response headers using Browser DevTools and Postman revealed that several important security headers are absent from API responses. The following were not present: Content-Security-Policy (CSP), X-Frame-Options, Strict-Transport-Security (HSTS), and X-Content-Type-Options.	
Evidence	Response headers observed: Content-Type, Access-Control-Allow-Origin: *, Cache-Control, Expires, Pragma. Missing: CSP, X-Frame-Options, HSTS, X-Content-Type-Options	
Business Impact	Missing security headers can expose consumers of this API to clickjacking, MIME-type sniffing attacks, and downgrade attacks if the API were embedded in a web application. The wildcard CORS header (Access-Control-Allow-Origin: *) allows any domain to call this API from browser-based JavaScript.	
Remediation	Add standard security headers to all API responses: set X-Content-Type-Options to nosniff, add HSTS for HTTPS enforcement, configure CORS to allow only trusted domains instead of the wildcard *, and add X-Frame-Options: DENY.	

F-06	No Authorization or Access Control Between Users	MEDIUM
OWASP Reference	API1:2023 – Broken Object Level Authorization (BOLA)	
Observation	There is no concept of authorization in the API. Any caller can access any user's data by simply changing the ID in the URL — for example, /users/1 through /users/10 are all equally accessible. There is no check to verify that the requesting party has permission to view a particular user's data. This is a classic BOLA (also called IDOR) vulnerability pattern.	
Evidence	GET /users/1, /users/2 ... /users/10 — all return 200 OK without any ownership or permission check	
Business Impact	In a production application, BOLA vulnerabilities are consistently ranked as the #1 most common and impactful API security issue (OWASP API Top 10). An attacker could enumerate all user records by incrementing the ID value, accessing data that should only belong to specific authenticated users.	
Remediation	Implement object-level authorization checks on every endpoint. Verify that the authenticated user has permission to access the requested resource before returning any data. Never expose sequential numeric IDs — use UUIDs instead to prevent easy enumeration.	

F-07	No Input Validation Feedback on Write Endpoints	LOW
OWASP Reference	API8:2023 – Security Misconfiguration	
Observation	A test POST request was sent to /posts with minimal and invalid body data. The API returned a 201 Created response without any validation error. The API accepts any input structure without enforcing required fields, data types, or input length limits. This indicates no server-side input validation is in place.	
Evidence	POST /posts with body {} → 201 Created (no validation error, no required field enforcement)	
Business Impact	Without input validation, a real API would be vulnerable to injection attacks, oversized payload attacks, and data integrity issues. Clients could submit garbage data that breaks downstream systems or storage layers.	
Remediation	Implement strict schema validation on all write endpoints. Use a validation library to enforce required fields, data types, and maximum lengths. Return clear 400 Bad Request errors with descriptive messages when validation fails.	

F-08	API Versioning Not Implemented	LOW
OWASP Reference	API8:2023 – Security Misconfiguration	
Observation	The API does not use versioning in its URL structure. All endpoints are accessed directly at the root path (e.g., /posts, /users) with no version prefix such as /v1/ or /api/v1/. There is no version header or version negotiation mechanism present in requests or responses.	
Evidence	Base URL: jsonplaceholder.typicode.com/users (no /v1/ or /api/ prefix)	
Business Impact	While not a direct security vulnerability, lack of versioning makes it harder to deprecate old, insecure versions of an API safely. In real APIs, versioning is important for rolling out security patches without breaking existing clients.	
Remediation	Structure API endpoints with version prefixes (e.g., /api/v1/users). This allows older, potentially insecure versions to be deprecated and retired without affecting clients using newer, more secure endpoints.	

F-09	Verbose Error Messages on Invalid Requests	LOW
OWASP Reference	API8:2023 – Security Misconfiguration	
Observation	When a request is made to a non-existent resource (e.g., /posts/9999), the API returns an empty object {} with a 404 status. While not overly verbose, the API does not suppress implementation details. In some frameworks, stack traces or server version info can leak through error responses.	
Evidence	GET /posts/9999 → 404 with response body: {}	
Business Impact	Verbose errors in production APIs can reveal server technology, framework versions, file paths, or database structure — all useful to an attacker during reconnaissance. Even minimal error information should follow a consistent, sanitized format.	
Remediation	Use a standard error response format for all error types. Never expose stack traces, internal paths, or framework details in API error responses. Return generic messages like 'Resource not found' rather than implementation-specific details.	

6 Risk Summary Table

The table below provides a consolidated view of all findings, their severity, OWASP category, and remediation priority.

ID	Finding	Severity	OWASP Category	Priority
F-01	No Authentication on Any Endpoint	HIGH	API2:2023	Immediate
F-02	Excessive Data Exposure – /users Endpoint	HIGH	API3:2023	Immediate
F-03	Email Addresses Exposed in /comments Endpoint	MEDIUM	API3:2023	Short-term
F-04	No Rate Limiting Detected	MEDIUM	API4:2023	Short-term
F-05	Missing Security Headers in HTTP Responses	MEDIUM	API7:2023	Short-term
F-06	No Authorization or Access Control Between Users	MEDIUM	API1:2023	Short-term
F-07	No Input Validation Feedback on Write Endpoints	LOW	API8:2023	Planned
F-08	API Versioning Not Implemented	LOW	API8:2023	Planned
F-09	Verbose Error Messages on Invalid Requests	LOW	API8:2023	Planned

2 HIGH	4 MEDIUM	3 LOW	9 TOTAL
-----------	-------------	----------	------------

7 Business Impact Analysis

This section explains what the identified risks would mean for a real business or SaaS product — written in non-technical language for stakeholders.

Data Privacy & Compliance Risk

The exposure of user details such as email addresses, phone numbers, home addresses, and GPS coordinates — with zero authentication — would be a serious GDPR violation in any real product. Regulators can issue fines of up to 4% of annual global turnover for such breaches. Beyond fines, the reputational damage of a data exposure incident can permanently affect customer trust.

Operational Risk – Service Availability

Without rate limiting, the API is wide open to request flooding. A single bad actor — or even a misconfigured client application — could send thousands of requests per second and bring down the service for legitimate users. For a SaaS product, downtime directly translates to lost revenue and customer churn.

Security Posture Risk

Missing security headers and wildcard CORS configurations weaken the overall security posture of the platform. Any web application consuming this API would inherit these weaknesses, potentially exposing end users to browser-based attacks.

Data Integrity Risk

The lack of input validation on write endpoints means that in a production system, clients could submit malformed, oversized, or malicious data without being rejected. This could corrupt databases, crash downstream services, or create injection vectors.

8 Remediation Recommendations

The recommendations below are prioritized by severity. Immediate actions should be addressed before any production deployment. Short-term and planned items should be scheduled into development sprints accordingly.

Immediate (High Priority)

- **Implement Authentication:** Add JWT or OAuth 2.0 authentication to all endpoints. Use an API gateway (e.g., Kong, AWS API Gateway) to enforce auth uniformly.
- **Fix BOLA / IDOR:** Add per-object authorization checks. Replace sequential integer IDs with UUIDs. Validate that the requesting user owns the resource being accessed.
- **Filter Response Data:** Return only the minimum fields needed per endpoint. Use response DTOs / serializers to control exactly what gets exposed.

Short-Term (Medium Priority)

- **Add Rate Limiting:** Configure per-IP and per-token rate limits. Return 429 with Retry-After header when limits are exceeded. Start with 60-100 requests/minute.
- **Add Security Headers:** Configure your web server or API gateway to add: X-Content-Type-Options: nosniff, Strict-Transport-Security, X-Frame-Options: DENY, and a proper Content-Security-Policy.
- **Restrict CORS:** Replace Access-Control-Allow-Origin: * with a whitelist of trusted frontend domains.
- **Mask Sensitive Fields:** Remove or mask email addresses, phone numbers, and geo coordinates from public endpoint responses.

Planned (Lower Priority)

- **Add Input Validation:** Use a schema validation library (e.g., Joi, Zod, Marshmallow) to enforce required fields, types, and length limits on all write endpoints.
- **Implement API Versioning:** Structure routes with a version prefix (/api/v1/) to support safe deprecation of older endpoint versions.
- **Standardize Error Responses:** Define a consistent error response format and ensure no stack traces or internal details are ever returned to clients.

9 OWASP API Security Top 10 Mapping

The findings in this report are mapped to the OWASP API Security Top 10 (2023 edition). This is the industry-standard framework for classifying and communicating API security risks. Reference: github.com/OWASP/API-Security

OWASP Category	Description	Finding(s)	Identified
----------------	-------------	------------	------------

API1:2023 – BOLA	Broken Object Level Authorization – accessing other users' objects	F-06	Yes
API2:2023 – Auth	Broken Authentication – missing or weak auth mechanisms	F-01	Yes
API3:2023 – BOPLA	Excessive data exposure / broken property-level auth	F-02, F-03	Yes
API4:2023 – Consumption	Unrestricted resource consumption / no rate limiting	F-04	Yes
API5:2023 – BFLA	Broken Function Level Authorization	–	Not tested
API6:2023 – SSRF	Unrestricted access to sensitive business flows	–	N/A
API7:2023 – SSRF	Server Side Request Forgery	–	N/A
API8:2023 – Misconfig	Security Misconfiguration (headers, CORS, errors)	F-05, F-07, F-08, F-09	Yes
API9:2023 – Inventory	Improper inventory management / versioning	F-08	Yes
API10:2023 – Unsafe Consumption	Unsafe consumption of APIs	–	N/A

10

Conclusion

This API security risk analysis was conducted on the JSONPlaceholder public demo API as part of Task 3 of the Future Interns Cyber Security Internship. The objective was to think and work like a real security consultant — not to break anything, but to systematically assess what's there and what's missing.

A total of 9 findings were identified across all major OWASP API Security Top 10 categories. The two High severity findings — no authentication and excessive data exposure — are the most critical. If this API were running in a real production environment, these issues alone would represent a significant security and compliance failure.

The medium severity findings around rate limiting, security headers, and CORS configuration are extremely common in real SaaS products. Many smaller startups overlook these during rapid development, only to face issues later. The low severity items, while not urgent, contribute to a stronger security posture over time.

This task provided practical experience in the kind of API audit work that security consultants and AppSec engineers do daily. The skills used here — endpoint mapping, header inspection, OWASP framework application, and business-friendly risk communication — are directly applicable to real-world security roles.

Key Takeaway: APIs are only as secure as the controls built around them. Authentication, authorization, rate limiting, and proper data filtering are not optional extras — they are the foundation of any secure API design. This analysis demonstrates how even a simple public API can surface a wide range of realistic security considerations when examined systematically.

1
1

References

- [1] **JSONPlaceholder – Official Site**
<https://jsonplaceholder.typicode.com>
- [2] **JSONPlaceholder – API Guide**
<https://jsonplaceholder.typicode.com/guide>
- [3] **OWASP API Security Top 10 (GitHub)**
<https://github.com/OWASP/API-Security>
- [4] **API Security Checklist by Shieldfy (GitHub)**
<https://github.com/shieldfy/API-Security-Checklist>
- [5] **Public APIs Collection (GitHub)**
<https://github.com/public-apis/public-apis>
- [6] **Postman – API Testing Tool**
<https://www.postman.com>
- [7] **Future Interns – Cyber Security Track**
<https://www.linkedin.com/company/future-interns/>

End of Report | API Security Risk Analysis – FUTURE_CS_03 | Future Interns Cyber Security Track